Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

`https://github.com/VenkatChowdary999/DeepLearning-Labs/upload/main/Lab-1`

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples

- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	**Purpose**	**Expected Inference**
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot.

8. Performance Tables

Training Summary

Dataset Size	1732
Train/Test Split	80-20
Learning Rate	0.01
Epochs	20
Final Weights	[-0.15641765 -0.1746633 -0.15518554 -0.02596029]
Final Bias	-0.09
Accuracy	0.9781818181818182
Precision	1.0
Recall	0.9212598425196851
F1-score	0.9590163934426229

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

10. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results

Table 2: Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	47	-0.072537	-0.064863	-0.064460	0.000199	-0.03
2	26	-0.082170	-0.093802	-0.051494	-0.005182	-0.03
3	34	-0.094676	-0.091400	-0.089055	-0.009565	-0.05
4	23	-0.110745	-0.095809	-0.106307	-0.013124	-0.04
5	27	-0.130964	-0.121796	-0.091128	-0.008771	-0.05
6	25	-0.139534	-0.128430	-0.104882	-0.006691	-0.04
7	19	-0.135270	-0.140369	-0.105269	-0.005157	-0.05
8	19	-0.133463	-0.144820	-0.104906	0.001333	-0.06
9	16	-0.134315	-0.131837	-0.123976	-0.011640	-0.06
10	26	-0.131687	-0.155810	-0.126006	0.011535	-0.06
11	25	-0.134304	-0.163344	-0.123422	0.004369	-0.07
12	26	-0.134814	-0.172105	-0.129577	0.001285	-0.07
13	21	-0.141143	-0.177991	-0.129746	0.005730	-0.08
14	21	-0.140347	-0.177649	-0.147387	0.006314	-0.07
15	22	-0.148292	-0.190448	-0.136416	0.000015	-0.07
16	16	-0.164249	-0.175786	-0.139463	-0.012613	-0.07
17	17	-0.149648	-0.171091	-0.157396	-0.018455	-0.08
18	15	-0.150822	-0.193664	-0.142668	0.008333	-0.07
19	15	-0.149548	-0.174279	-0.163977	0.006658	-0.08
20	17	-0.156418	-0.174663	-0.155186	-0.025960	-0.09

- Generated Plots with Interpretation

  Interpretation: this plot shows how each feature is distributed in the dataset it identifies outliers skewness

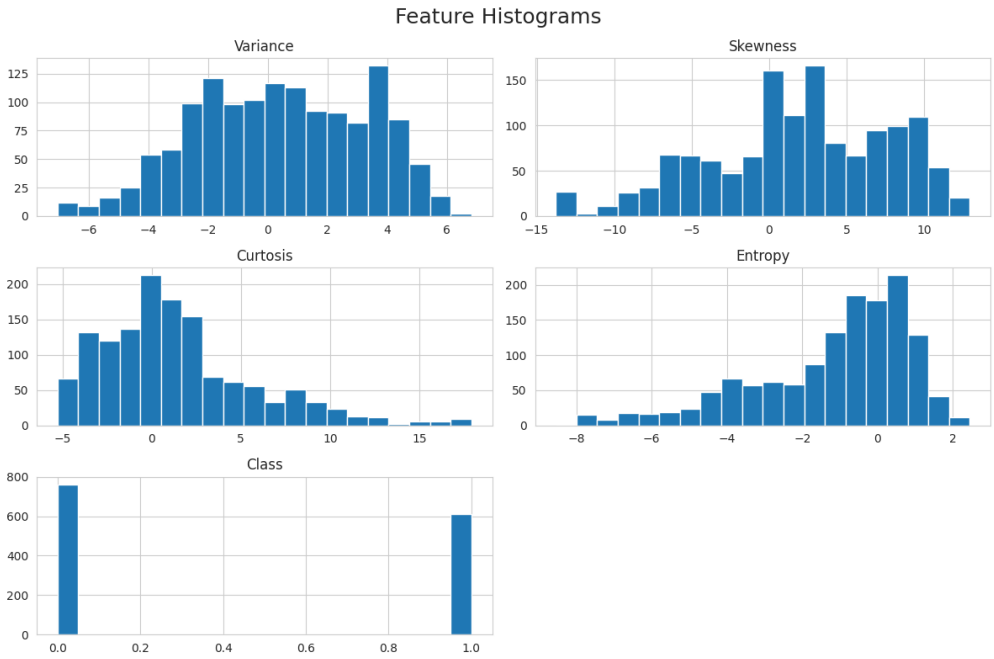

Figure 1: Histogram of Features

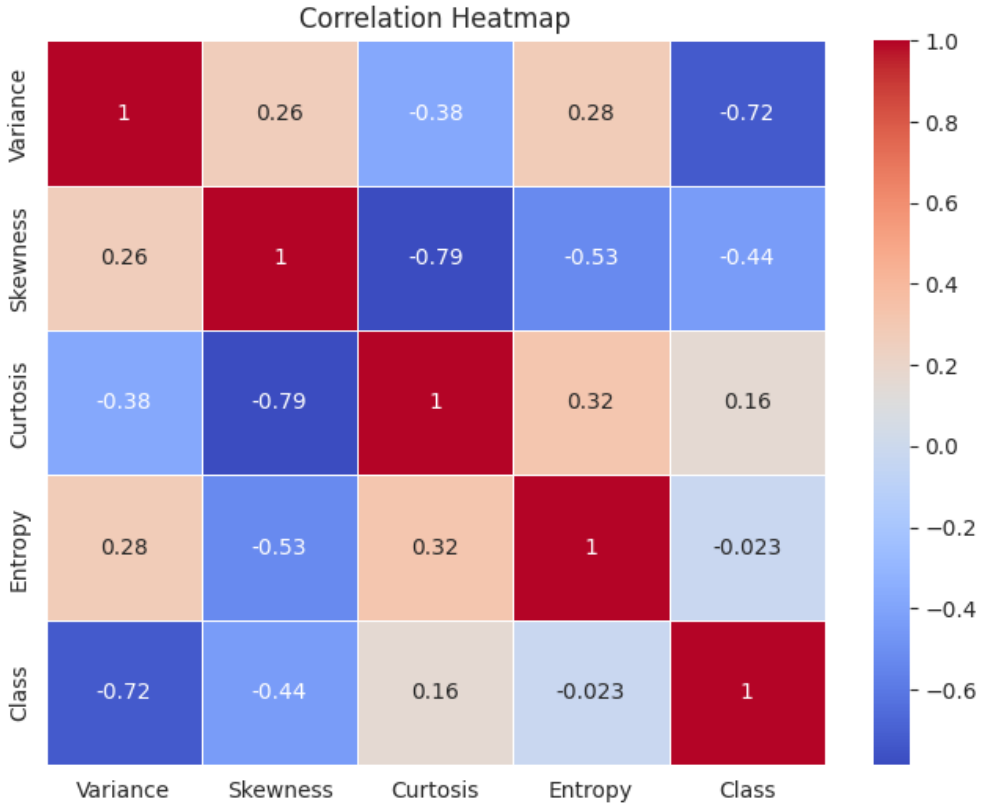

Figure 2: corelation heatmap

Interpretation: This fig shows the corelation bw input features and target class , we can see that higher the corelation higher the contribution to classification

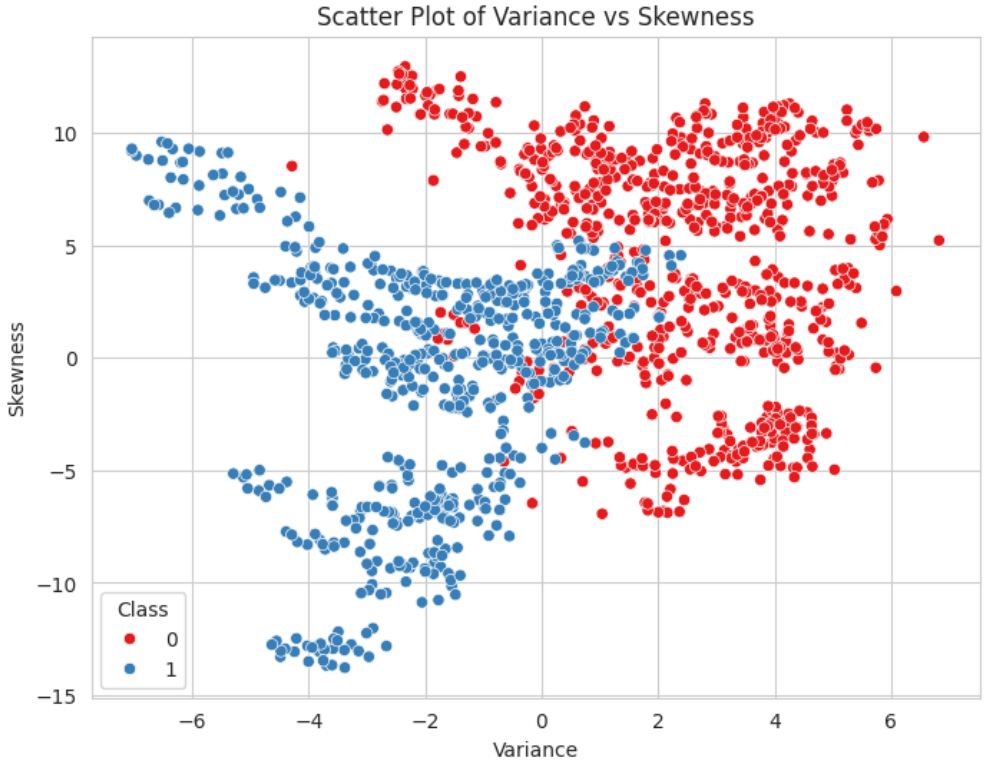

Figure 3: Scatter Plot

Interpretation: this plot shows the relationship bw these features and by analysing the plot we can see that the dataset is approximately lineary seperable as there is good seperatrion

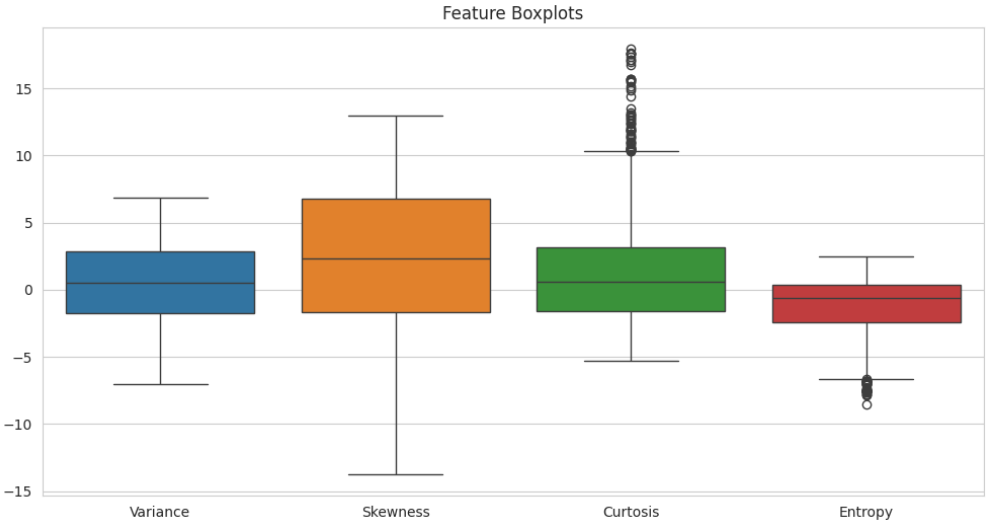

Figure 4: Boxplot

Interpretation: The box plot shows the spread of features and identifies the presence of any outliers in the data]

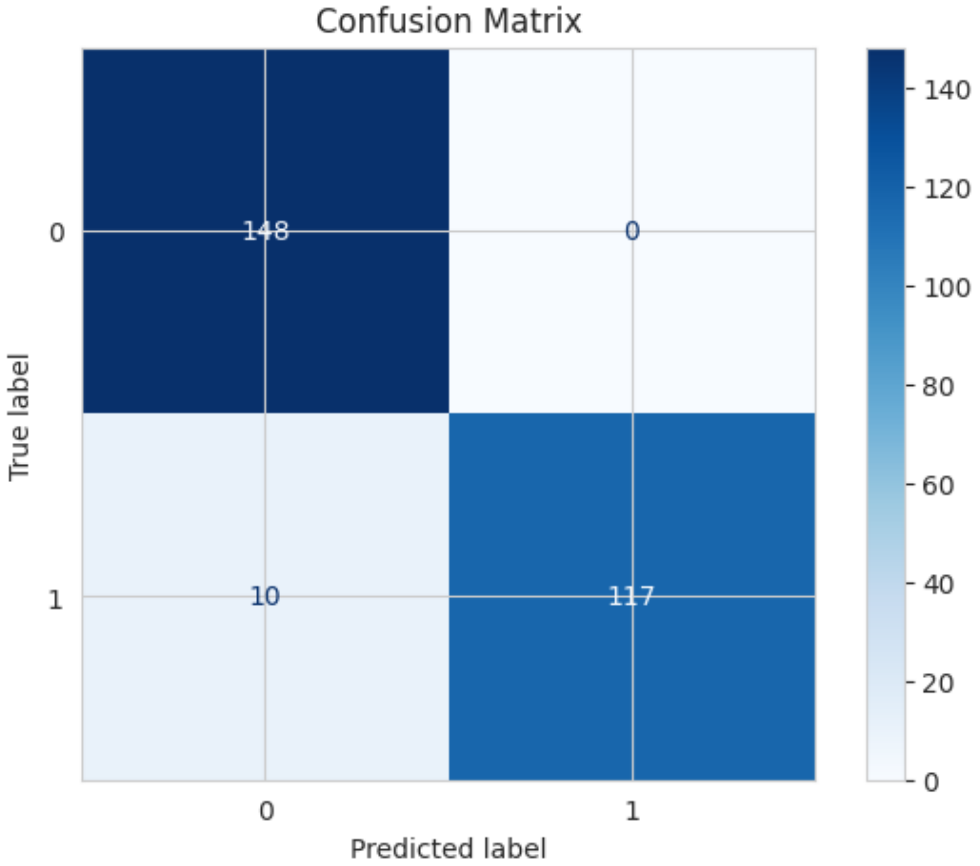

Figure 5: Confusion matrix

Interpretation: the confusion matrix shows the results of the performance and high values in the diagonal imply accurate classification

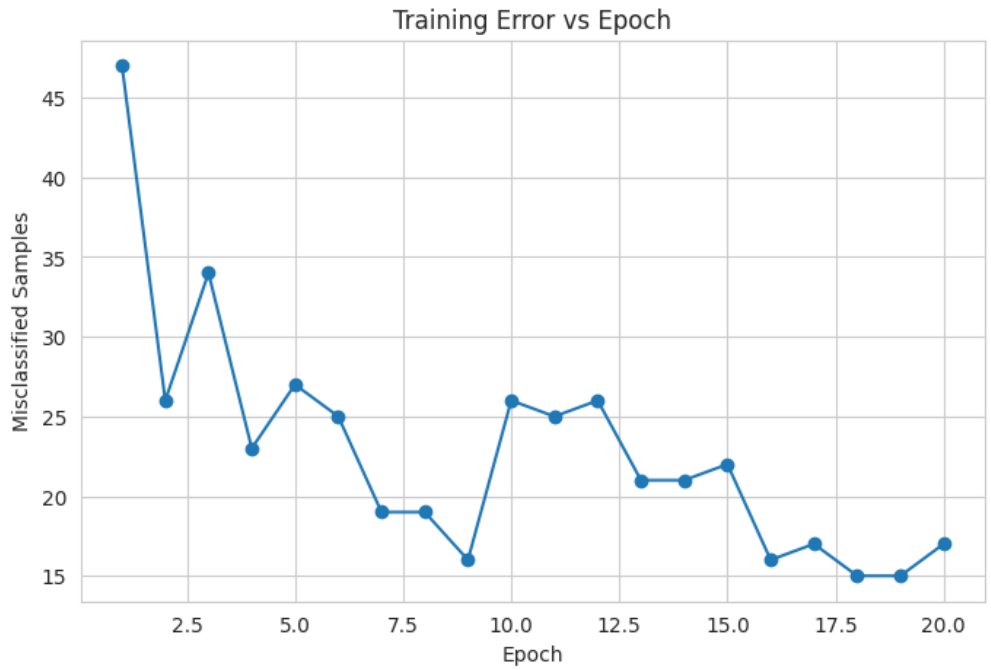

Figure 6: Error VS Epoch

Interpretation: as epoch increases the no of missclassified samples decreases showing the convergence of algorithm

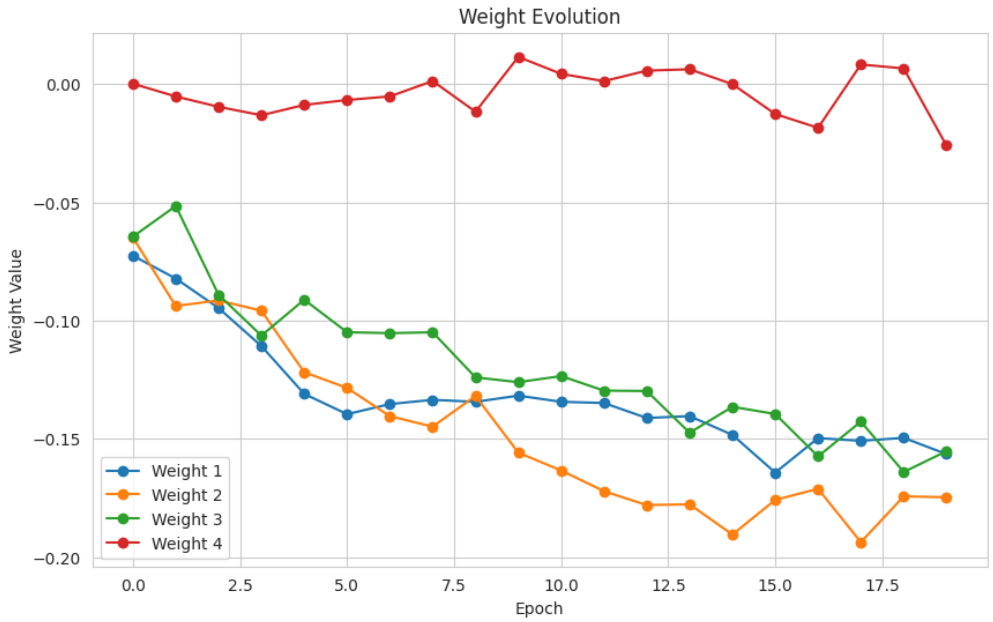

Figure 7: Weight evolution

Interpretation: weight values are updated during training and generally converge to a stable values a sthe algorithm proceeds

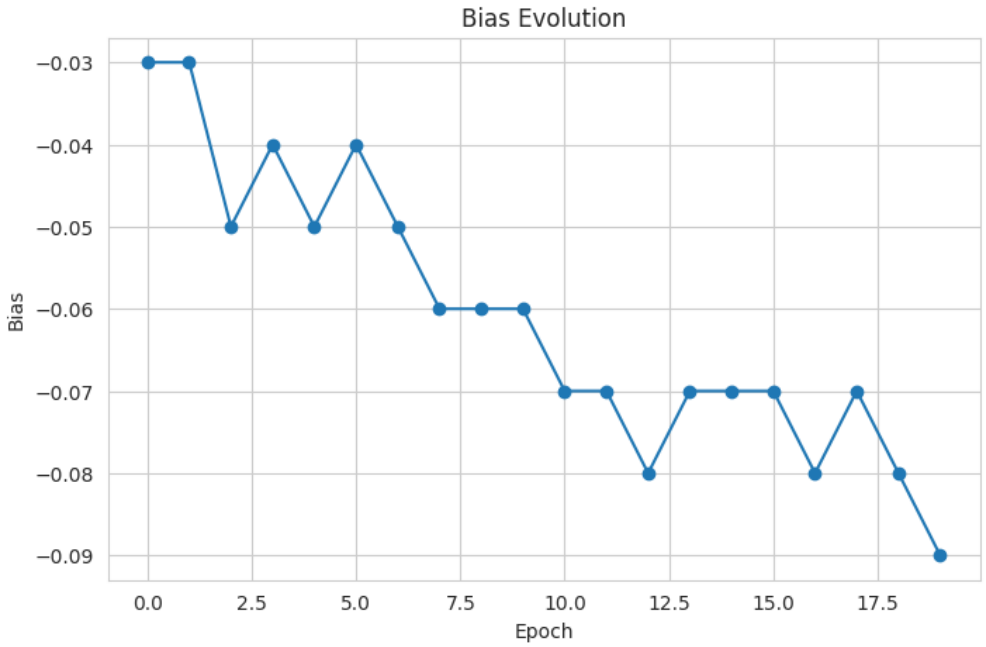

Figure 8: bias Evolution

Interpretation: the bias value changes during training indicating successful training

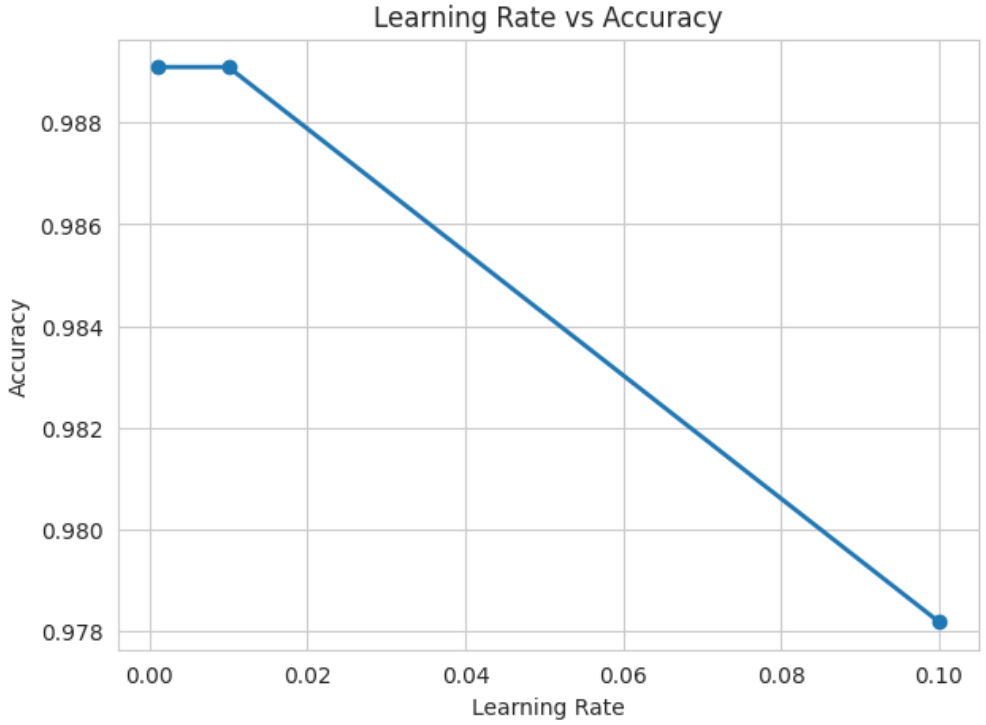

Figure 9: Learning Rate vs accuracy

Interpretation: this graph shows accuracy for different learning rates we observe moderate

learning rates produce stable results

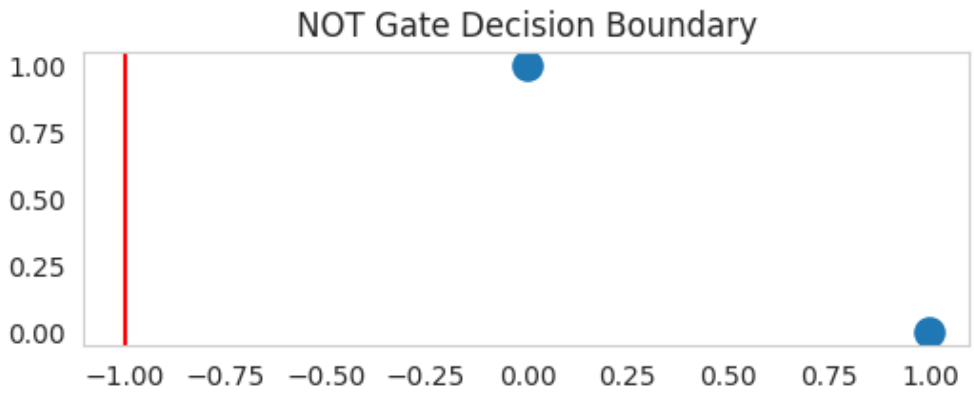

Figure 10: Not Gate decision boundary

Interpretation: the percepton learns a threshold that correctlu classifies not gate

- Performance Tables
- Discussion The Perceptron model successfully learned the classification of banknotes by updating the weights and bias for multiple epochs. As training continued, the number of classification errors decreased, indicating that the model was able to learn a good linear decision boundary. The performance metrics of accuracy, precision, recall, and F1-score showed that the model performed well on the dataset. The experiment also emphasized the significance of feature representation and iterative learning for improving prediction accuracy. But the Perceptron can only handle linearly separable data and may not work well with more complex data sets. Overall, this experiment gave a clear understanding of the Perceptron learning algorithm and its use in binary classification problems.
- Conclusion The Perceptron algorithm was able to classify banknotes as authentic or counterfeit using their respective features. The model improved by adjusting weights over several epochs, reducing classification errors and increasing its prediction accuracy. The results showed that the Perceptron performs well on linearly separable problems and provides a simple yet efficient method for binary classification. In summary, this experiment provided insight into the behavior of the Perceptron algorithm, the evolution of the decision boundary during training, and the evaluation of performance metrics, including accuracy, precision, recall, and F1-score. It also highlighted the importance of training and feature representation for dependable predictions.
- References

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

5. UCI Machine Learning Repository – Banknote Authentication Dataset.

6. Scikit-learn Documentation: Perceptron.